

AI Agents for Love Letter: Reasoning, Belief Search, and Reinforcement Learning under Hidden Information

Yanqi Zhang

Yueqian Ma

Yunyang Liu
CS181 Final Project

Abstract

This project studies *Love Letter* as a compact stochastic, partially observable, adversarial game. We implement a reusable Python game environment and compare several agent families from the CS181 course: random play, handcrafted heuristics, symbolic logic, approximate Bayesian belief search, expectimax, Monte Carlo tree search, tabular Q-learning, and feature-based approximate Q-learning. The central implementation question is not simply whether an agent can win against a weak baseline, but whether it does so under the same hidden information available to a real player. Therefore, the project separates fair hidden-information agents from perfect-information oracle baselines. Pilot experiments show that card-counting heuristics, logic constraints, and Q-learning with heuristic priors and reward-shaped approximate Q-learning consistently outperform random play, while belief search is harder to tune and has high variance at small sample sizes. The result is a complete experimental platform for studying uncertainty, search, and learning in a small but nontrivial imperfect-information game.

1 Introduction

Love Letter is a small card game with only 16 cards, but it contains the features that make many AI problems difficult: private hands, a hidden burned card, stochastic draws, adversarial targeting, and actions that reveal partial information. This makes it a useful course project because the full rules are implementable within a semester, while the decision problem still requires reasoning under uncertainty.

Our proposal framed the project around four agent families: heuristic agents, Bayesian agents, expectimax search, and reinforcement learning with MCTS. The current implementation follows that plan and refactors the branch work into a single package. In addition, we explicitly distinguish two kinds of search:

- **Fair hidden-information search**, where the agent samples hidden states from an observation-derived belief and does not directly inspect the opponent’s true hand.

Table 1: Base Love Letter deck used by the environment.

Card	Value	Count	Effect summary
Guard	1	5	Guess a non-Guard card.
Priest	2	2	Privately view a hand.
Baron	3	2	Compare hands; lower card loses.
Handmaid	4	2	Gain temporary protection.
Prince	5	2	Force discard and re-draw.
King	6	1	Swap hands.
Countess	7	1	Forced with King or Prince.
Princess	8	1	Lose if discarded.

- **Oracle search baselines**, where the agent searches from a cloned true environment. These baselines measure an upper bound for planning strength, but they are not directly comparable to real hidden-information play.

This distinction is important for soundness. If an agent sees the full environment state, high win rates are less meaningful because they may come from illegal information. The report therefore presents oracle agents as diagnostic baselines rather than as the main claim.

2 Game Model

The base deck is shown in Table 1. Each turn consists of drawing one card, then playing one of the two cards in hand. A round ends when all but one player are eliminated or when the deck is exhausted and the surviving players compare their remaining cards.

We model a complete state as

$$s = (s_{\text{pub}}, s_{\text{priv}}), \quad (1)$$

where s_{pub} includes discard piles, eliminated players, protection flags, turn index, current player, and deck size,

while s_{priv} includes private hands, the hidden set-aside card, and the exact deck order. For player i , the observation function is

$$o_i = O_i(s) = (\text{public view}, \text{own hand}, \text{private events}_i). \quad (2)$$

Agents receive o_i and the legal action set $A(o_i)$.

The transition model is mixed:

- Card effects such as Guard, Baron, King, and Princess are deterministic once the hidden state is fixed.
- Draws are stochastic from the agent’s perspective because the deck order and hidden card are unknown.

For reinforcement learning, terminal reward is implemented as +1 for a win and −1 for a loss. This differs slightly from the proposal’s 0 loss reward, but it provides a clearer learning signal and separates losing terminal states from neutral nonterminal transitions.

3 Environment Implementation

The environment is implemented in `loveletter_ai.environment`. `LoveLetterEnv` owns the true state, legal action generation, seeded randomness, and card-effect resolution. The true state is represented by `GameState`; the hidden-information-safe player view is represented by `Observation`. This boundary is the main engineering device that keeps agents fair.

Important implementation choices include:

- **Deterministic reproducibility.** Each round uses an explicit seed. This makes tournament and ablation results repeatable.
- **Rule-complete legal actions.** Legal action generation handles Countess forced play, Guard guess restrictions, Handmaid protection, and Prince self-targeting.
- **Public/private event history.** Priest observations are private to the viewer, while public events such as discards and eliminations are visible to all agents.
- **Search cloning.** The environment supports deep cloning for expectimax and MCTS rollouts. The fair search agents overwrite hidden fields with belief samples before rollouts.

The evaluation layer in `loveletter_ai.evaluation` provides `run_round`, `evaluate_pair`, and `evaluate_pair_symmetric`. Symmetric evaluation runs both seat assignments, reducing first-player and seed bias.

4 Agents

4.1 Random and Heuristic Agents

The random agent samples uniformly from legal actions and serves as the baseline. The naive heuristic agent scores actions with simple rules such as protecting the

Princess, preferring Guard guesses, and avoiding Baron when the retained card is weak.

The improved heuristic and greedy agents add card-count awareness. From an observation o , the agent estimates unseen card multiplicities

$$C_o(c) = C_{\text{deck}}(c) - C_{\text{own}}(c) - C_{\text{discard}}(c). \quad (3)$$

This lets it avoid impossible Guard guesses and estimate opponent hand values. The action score has the general form

$$H(o, a) = w_r \cdot \text{retained}(o, a) + w_t \cdot \text{target}(o, a) + w_g \cdot \text{guess}(o, a) + w_s \cdot \text{safety}(o, a). \quad (4)$$

The weights are handcrafted in code rather than learned. This makes the policy easy to inspect and provides a strong baseline for the more advanced agents.

4.2 Logic Agent

The logic agent maintains a persistent knowledge base of possible opponent cards. The set for opponent j is initialized as all card types and updated by public and private evidence:

- visible own cards and discard piles are removed;
- Priest observations collapse a target’s possible set to one card;
- failed Guard guesses remove the guessed card;
- King swaps reset affected hand facts because ownership changes.

If the agent holds a Guard and the knowledge base has a singleton target set, it makes the guaranteed guess. Otherwise it falls back to the improved heuristic policy.

4.3 Belief Expectimax

The Bayesian component builds an approximate posterior over hidden states from the observation and logic constraints. It is approximate because it does not enumerate all histories or model opponent policies. Instead, it samples opponent hands, the burned card, and deck order without replacement from remaining card counts:

$$b_i(h \mid o_i) \propto \mathbf{1}[h \text{ is consistent with } o_i] \prod_c C_o(c). \quad (5)$$

Belief expectimax evaluates a root action by sampling hidden states $h^{(1)}, \dots, h^{(K)}$ and averaging the expectimax value:

$$\hat{Q}(o_i, a) = \frac{1}{K} \sum_{k=1}^K V_{\text{exp}}(s_{\text{pub}}, h^{(k)}, a). \quad (6)$$

Opponent actions are treated as chance nodes with a uniform policy. Draw nodes are enumerated by card multiplicity when possible.

4.4 MCTS

The oracle MCTS agent uses UCT over cloned complete states:

$$\text{UCT}(v') = \frac{Q(v')}{N(v')} + c\sqrt{\frac{\log N(v)}{N(v')}}. \quad (7)$$

Rollouts use an improved heuristic for the root player and random play for opponents.

The fair **BeliefMCTSAgent** performs root information-set search. Each simulation samples a plausible hidden state from the belief, evaluates one root action, and updates root-level action statistics. This is less powerful than a full information-set MCTS tree, but it avoids directly reading hidden opponent cards and is therefore aligned with the project setting.

4.5 Q-Learning

The first Q-learning agent uses tabular state-action values over compact observation and action keys. The update rule is

$$Q(o, a) \leftarrow Q(o, a) + \alpha \left[r + \gamma \max_{a'} Q(o', a') - Q(o, a) \right]. \quad (8)$$

Because the Love Letter observation space is sparse, the agent combines learned Q-values with a heuristic prior:

$$\text{score}(o, a) = Q(o, a) + \lambda H_{\text{norm}}(o, a). \quad (9)$$

The improved training loop uses linearly decaying exploration, per-state learning-rate decay, mixed opponents, and small reward shaping terms. The mixed opponent pool samples Random, Naive, Improved, and self-play opponents so that the policy does not overfit to one behavior distribution. The shaping terms give dense feedback for tactically clear events: eliminating an opponent, being eliminated, discarding Princess, protecting Princess with Handmaid, and making an informative Priest move.

We also implement an approximate Q-learning agent with a linear value model

$$Q(o, a) = \sum_k w_k f_k(o, a). \quad (10)$$

The features include retained card value, normalized heuristic score, Guard guess probability, impossible Guard guesses, Baron value edge, Prince Princess-discard probability, and Princess safety indicators. Tactically monotone features are constrained to keep sensible signs, for example discarding Princess remains non-positive and protecting Princess remains non-negative. This makes approximate Q-learning a stable residual correction over the heuristic policy rather than a noisy replacement for it.

5 Experiments

5.1 Protocol

All reported pilot experiments use two-player rounds. For each pair, we use symmetric evaluation: agent A plays as

Table 2: Fair hidden-information agents vs Random, 20 games per seat.

Agent	W-L-D	Win rate	95% CI
Naive	27-13-0	67.5%	52.0–79.9
Logic	31-9-0	77.5%	62.5–87.7
Greedy	31-9-0	77.5%	62.5–87.7
Improved	31-9-0	77.5%	62.5–87.7
BeliefExpectimax	26-14-0	65.0%	49.5–77.9
BeliefMCTS	30-10-0	75.0%	59.8–85.8
Tabular Q+prior	31-9-0	77.5%	62.5–87.7
Approx Q+shaping	31-9-0	77.5%	62.5–87.7

player 0 for n seeds, then as player 1 for another n seeds. Win rate is the fraction of all $2n$ rounds won by the first listed agent. Confidence intervals are Wilson 95 percent intervals.

The current report tables were generated with:

```
python test_agent/run_ablation_experiments.py
20 outputs/report_experiments quick
```

Thus each row contains 40 games. These results are suitable for development analysis and presentation drafts, but the final submission should rerun the full profile with a larger sample size.

5.2 Fair Hidden-Information Results

Table 2 compares fair hidden-information agents against Random. The strongest simple agents are the logic, greedy, and improved heuristic agents, each reaching 77.5 percent in this pilot run. The repaired RL agents now match that tier: tabular Q with a heuristic prior reaches 77.5 percent, and approximate Q with shaping also reaches 77.5 percent. BeliefMCTS improves over the earlier belief-search result, while BeliefExpectimax remains more sensitive to sampling noise.

5.3 Ablation Results

Table 3 summarizes the main ablations. Three patterns are important. First, pure tabular Q-learning is weak because the state-action space is sparse. Second, adding a heuristic prior recovers most of the lost performance, and approximate Q-learning with shaping gives a small further gain. Third, more computation does not automatically improve a hidden-information search agent at small sample size: extra samples can increase variance if the belief model or rollout policy is weak.

6 Analysis

The main observation is that the fair agents do not usually exceed 80 percent against Random in the current

Table 3: Pilot ablations. Each row is evaluated against Random with 20 games per seat.

Family	Setting	W-L-D	Win rate	95% CI
BeliefExpectimax samples	samples=1	30-10-0	75.0%	59.8–85.8
BeliefExpectimax samples	samples=4	29-11-0	72.5%	57.2–83.9
BeliefExpectimax samples	samples=8	30-10-0	75.0%	59.8–85.8
BeliefExpectimax depth	depth=1	27-13-0	67.5%	52.0–79.9
BeliefExpectimax depth	depth=2	27-13-0	67.5%	52.0–79.9
BeliefMCTS simulations	simulations=4	26-14-0	65.0%	49.5–77.9
BeliefMCTS simulations	simulations=12	22-18-0	55.0%	39.8–69.3
BeliefMCTS simulations	simulations=24	27-13-0	67.5%	52.0–79.9
Q-learning variants	heuristic only	30-10-0	75.0%	59.8–85.8
Q-learning variants	pure tabular Q	24-16-0	60.0%	44.6–73.7
Q-learning variants	tabular Q+prior	30-10-0	75.0%	59.8–85.8
Q-learning variants	approx Q+shaping	31-9-0	77.5%	62.5–87.7
Approx Q episodes	episodes=0	29-11-0	72.5%	57.2–83.9
Approx Q episodes	episodes=500	30-10-0	75.0%	59.8–85.8
Approx Q episodes	episodes=1000	30-10-0	75.0%	59.8–85.8
Approx Q episodes	episodes=5000	31-9-0	77.5%	62.5–87.7
Oracle Expectimax	depth=1	30-10-0	75.0%	59.8–85.8
Oracle Expectimax	depth=2	33-7-0	82.5%	68.0–91.2
Oracle MCTS	simulations=4	32-8-0	80.0%	65.2–89.5
Oracle MCTS	simulations=12	32-8-0	80.0%	65.2–89.5

pilot setting. This is reasonable. Love Letter has high inherent variance: an early Guard hit, Princess draw, or Prince discard can decide a round before a policy’s long-term advantage has time to show. Because each row currently contains only 40 rounds, the confidence intervals are wide.

The most reliable improvement comes from eliminating clearly bad actions: avoiding impossible Guard guesses, respecting the Countess rule, protecting a Princess with Handmaid, and using remaining-card counts for target decisions. That explains why the improved heuristic, greedy, and logic agents are strong relative to their complexity.

Belief search is conceptually more aligned with the hidden-information game, but its current implementation is still approximate. It samples plausible states, assumes simple opponent behavior, and uses relatively shallow search. When the belief is noisy, deeper search can amplify a bad determinization rather than fix it. This explains why the sample and depth ablations do not yet show monotonic improvement.

The Q-learning ablation confirms the diagnosis from development. Pure tabular Q-learning is not enough: it reaches only 60 percent in the pilot table. Adding the heuristic prior recovers the baseline to 75 percent, while the approximate Q-learning agent with reward shaping reaches 77.5 percent. This does not prove a large statistically significant RL advantage yet, because the confidence intervals are wide, but it gives a much cleaner story: each

repair addresses a concrete weakness of the previous RL version.

7 Current Shortcomings

Compared with the reference report, this project is strongest in environment soundness and hidden-information framing, but weaker in final experimental scale. The main remaining issues are:

- **Sample size.** The current report tables use pilot-scale experiments. A final report should use the full ablation profile and a larger symmetric tournament.
- **Tournament matrix.** The proposal promised a full agent tournament. The code supports pairwise evaluation, but the final report should include an all-pairs matrix similar to the reference report’s Sente and Gote tables.
- **Belief model.** The Bayesian agent is an approximate consistent-state sampler, not a full posterior over histories.
- **RL maturity.** The repaired RL agent now has meaningful ablations, but the final report should still include larger learning curves before claiming a statistically significant advantage.
- **Two-player focus.** The engine supports 2–4 players, but the reported experiments use two-player rounds for controlled comparison.

8 External Resources and Tools

No external Love Letter engine code was copied. The game engine, agents, and evaluation scripts are implemented in Python within this repository. External resources and tools used are:

- **Python standard library:** dataclasses, enum, random, copy, collections, and pathlib are used for state structures, seeded stochastic transitions, cloning, and counters.
- **pytest:** used for unit and integration tests covering setup, hidden-information rendering, belief sampling, agent legality, and match completion.
- **Tectonic/LaTeX:** used to compile this report from the LaTeX source.
- **Codex AI coding assistant:** used as a development assistant for branch integration, refactoring, testing, report drafting, and experiment organization. The team should review all code and explanations before final submission.
- **Love Letter base rules:** used as the game specification for the 16-card deck and card effects. No third-party implementation was used.

9 Conclusion

The project delivers a coherent Love Letter AI platform with a hidden-state environment, multiple agent families, fair and oracle search baselines, and reproducible evaluation scripts. The strongest course connection is the combination of partial observability, belief modeling, adversarial search, and reinforcement learning. To reach a high final grade, the main repair is not to add another complicated agent, but to strengthen the evidence: run a larger symmetric tournament, report confidence intervals, include ablations, and make the hidden-information fairness distinction explicit in both the report and presentation.

References

- [1] Seiji Kanai. *Love Letter*. Alderac Entertainment Group, base game rules.
- [2] pytest documentation. <https://docs.pytest.org/>
- [3] Tectonic Typesetting System. <https://tectonic-typesetting.github.io/>